

TP3

STM32F446RE CAN Communication

via MCP2515 CAN Transceiver

Filière:

2ème Années Mécatronique & Technologies Automobiles

Supervised by:

Pr. Wissam JENKAL

Mr. Rachid AMENZOUY

0 Contents

1	Project Overview	2
1.1	CAN Bus Key Concepts	2
2	Hardware Requirements	2
3	Pin Mapping and Wiring	2
3.1	Wiring Diagram	3
4	MCP2515 Driver	3
5	STM32CubeMX Configuration	3
5.1	Master Board Configuration	4
5.1.1	SPI1 — Master	4
5.1.2	GPIO — Master	4
5.1.3	NVIC — Master	5
5.1.4	Complete <code>main.c</code> — Master	5
5.2	Slave Board Configuration	7
5.2.1	SPI1 — Slave	7
5.2.2	GPIO — Slave	8
5.2.3	NVIC — Slave	8
5.2.4	Complete <code>main.c</code> — Slave	8
6	Communication Protocol	11
6.1	Frame Description	11
6.2	Transaction Sequence	11
7	Project Validation	12

1 Project Overview

This guide implements **CAN bus** communication between two STM32F446RET6 boards using the **MCP2515** SPI-to-CAN controller as an external transceiver:

- **Master board** periodically transmits a CAN frame with identifier 0x100 and payload byte 0xAA every 3 seconds, instructing the slave to activate its LED and buzzer.
- **Slave board** listens on the CAN bus using interrupt-driven reception (EXTI on the MCP2515 INT pin). When the expected frame arrives, it blinks an LED and sounds a buzzer three times.

1.1 CAN Bus Key Concepts

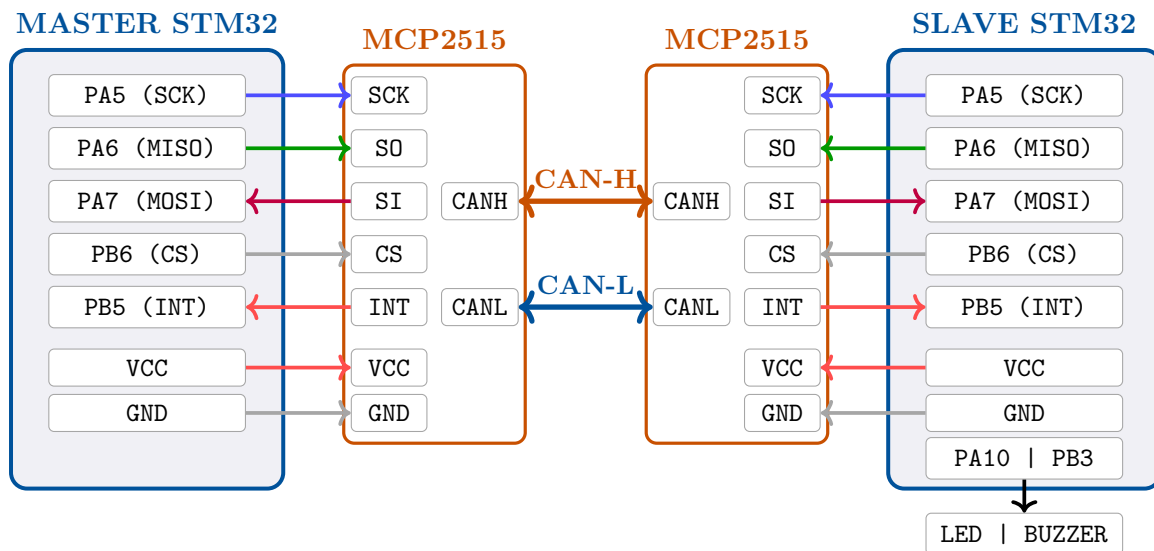
Term	Meaning
CAN-H / CAN-L	Differential pair carrying the bus signal
Standard frame	11-bit identifier, up to 8 data bytes
Arbitration ID	Priority identifier; lower value = higher priority
DLC	Data Length Code — number of data bytes (0–8)
Dominant / Recessive	Logic 0 (dominant) always wins arbitration
ACK slot	Any receiver drives dominant to acknowledge the frame
MCP2515	Standalone SPI-to-CAN controller with integrated transceiver

2 Hardware Requirements

Component	Details
STM32F446RET6	× 2 — Master and Slave boards
MCP2515 module	× 2 — one per board (includes TJA1050 transceiver)
LED	Connected to slave GPIO output (PA10)
Buzzer (active)	Connected to slave GPIO output (PB3)
Jumper wires	SPI lines + CAN bus (CAN-H / CAN-L) + power
3.3 V / 5 V power supply	Common GND between all boards and modules

3 Pin Mapping and Wiring

3.1 Wiring Diagram



4 MCP2515 Driver

The MCP2515 is controlled via SPI using a dedicated driver (`mcp2515.c` / `mcp2515.h`). Both projects must include this driver. The key API functions used in this application are summarised below.

Function	Description
<code>MCP2515_Init()</code>	Resets the MCP2515, configures the bit-rate to 500 kbps , and switches to Normal mode. Must be called after a short power-on delay.
<code>MCP2515_SendFrame(CAN_Frame_t *frame)</code>	Loads the frame into TXB0 and requests transmission. Returns when the frame has been queued.
<code>MCP2515_CheckReceive()</code>	Returns non-zero if a frame is waiting in RXB0 (polls the CANINTF register).
<code>MCP2515_ReadFrame(CAN_Frame_t *frame)</code>	Reads the received frame from RXB0 into the caller's <code>CAN_Frame_t</code> structure and clears the interrupt flag.

Navigate to **Core** → **Src** folder in your project, then past the **mcp2515.c** file.
 Navigate to **Core** → **Inc** folder in your project, then past the **mcp2515.h** file.

5 STM32CubeMX Configuration

Both boards share an *identical* CubeMX configuration because the MCP2515 is controlled entirely over SPI — the STM32's built-in CAN peripheral is **not** used.

5.1 Master Board Configuration

Create a new STM32CubeMX or STM32CubeIDE project targeting **STM32F446RETx**.

5.1.1 SPI1 — Master

Navigate to **Connectivity** → **SPI1** and apply:

Parameter	Value
Mode	Full-Duplex Master
Data Size	8 Bits
Clock Polarity	Low (CPOL = 0)
Clock Phase	1 Edge (CPHA = 0)
NSS	Software
Baud Rate Prescaler	16
First Bit	MSB First
CRC Calculation	Disabled

CubeMX will automatically assign **PA5** (SCK), **PA6** (MISO), and **PA7** (MOSI) in their alternate-function modes.

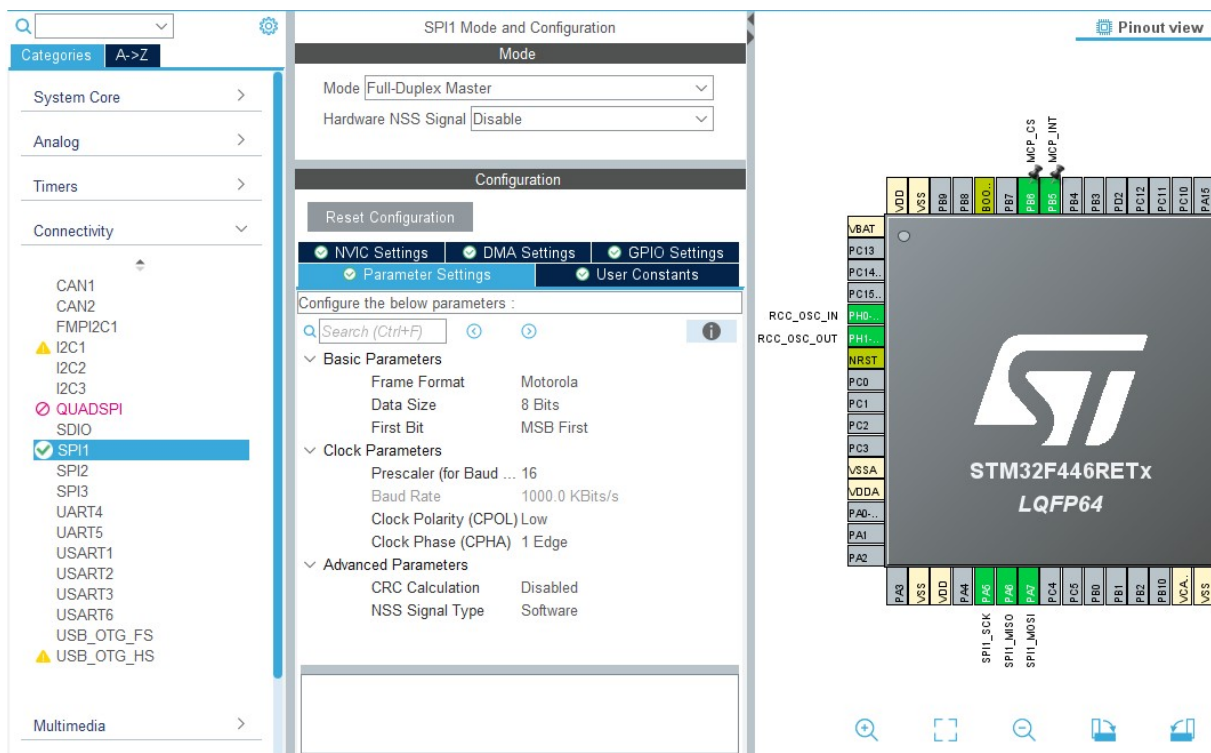


Figure 1: SPI1 Master Configuration.

5.1.2 GPIO — Master

Navigate to **System Core** → **GPIO** and configure:

Pin	Label	Mode
PB6	MCP_CS	GPIO Output
PB5	MCP_INT	GPIO EXTI5

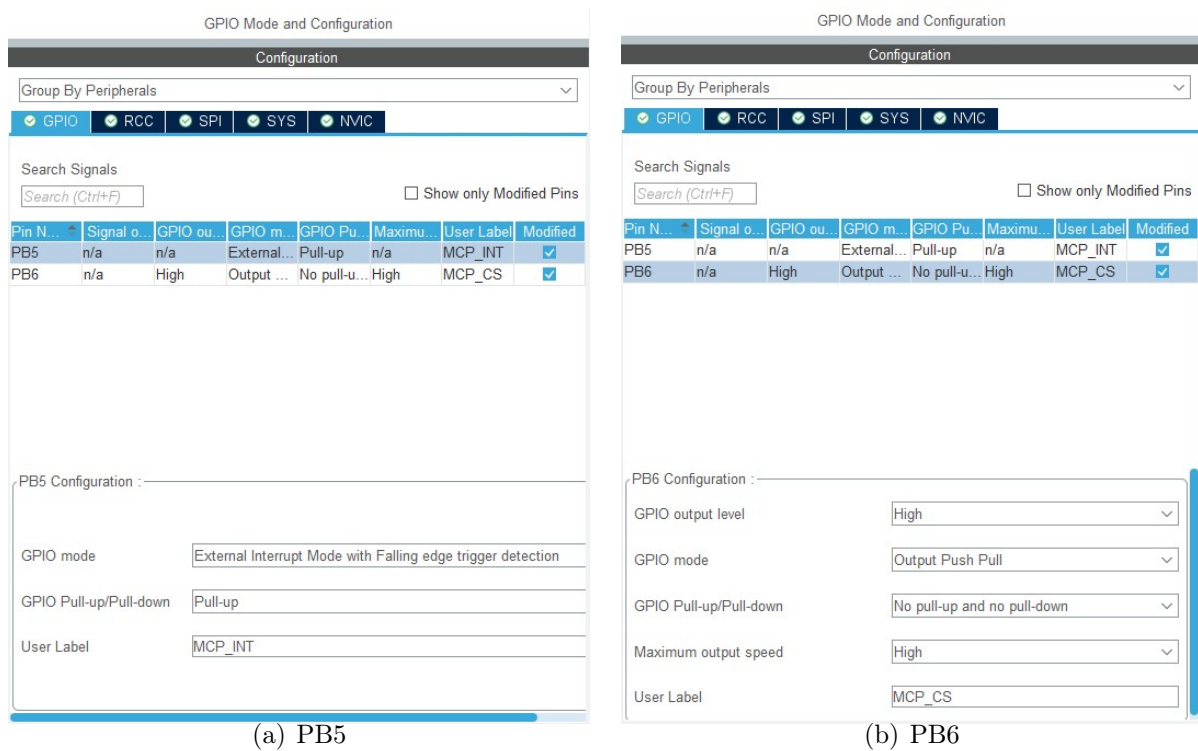


Figure 2: GPIO Configuration.

5.1.3 NVIC — Master

Navigate to **System Core** → **NVIC** and enable:

- **EXTI line[9:5] interrupts**, then set priority to 0, 0.

Save the configuration: After finishing the configuration click **Ctrl + S**.

5.1.4 Complete main.c — Master

Open the Master **main.c** file and complete the application:

```

1  /* Includes
   *-----*/
2  #include "main.h"
3
4  /* USER CODE BEGIN Includes */
5  #include "mcp2515.h"
6  /* USER CODE END Includes */

```

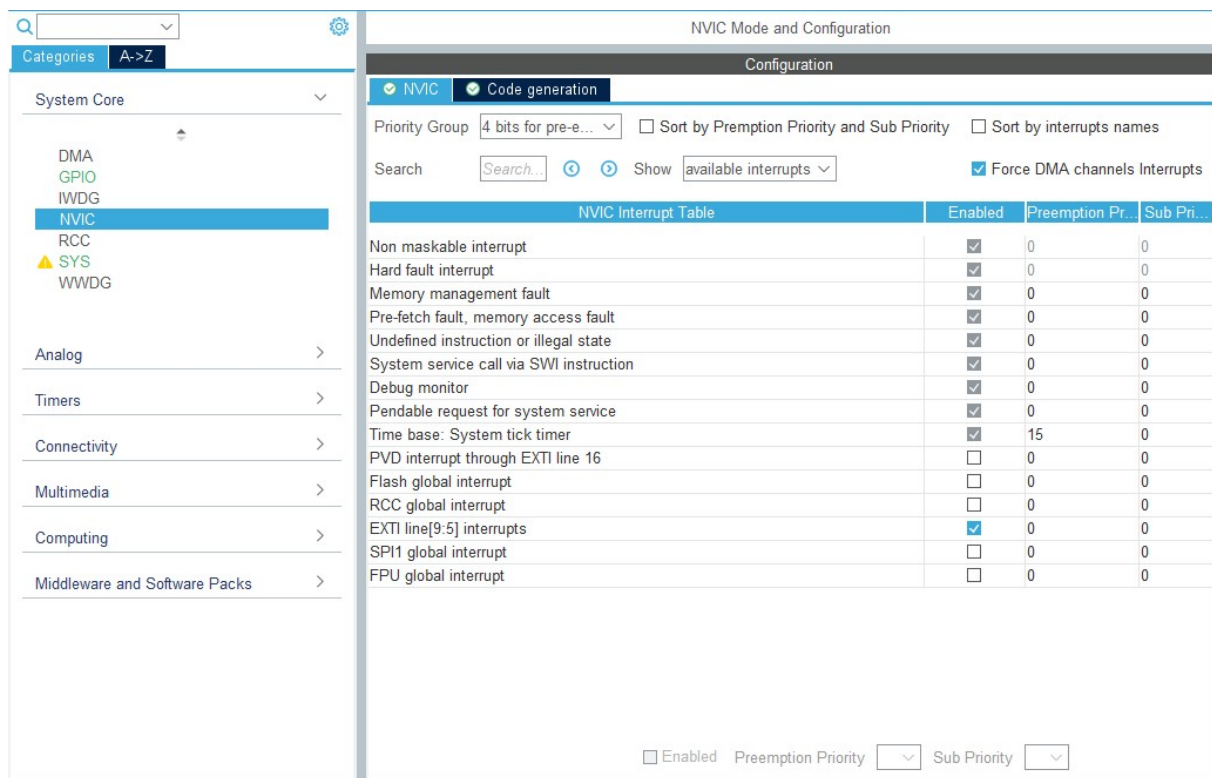


Figure 3: NVIC Configuration.

```

7
8  /* Private variables
9  ----- */
10
11 SPI_HandleTypeDef hspi1;
12
13 /* USER CODE BEGIN PV */
14 CAN_Frame_t tx_frame;
15 /* USER CODE END PV */
16
17 int main(void)
18 {
19     /* USER CODE BEGIN 2 */
20
21     // Let the MCP2515 stabilise after power-on
22     HAL_Delay(100);
23
24     // Initialise MCP2515: reset, configure 500 kbps, switch to
25     // Normal mode
26     MCP2515_Init();
27
28     // Prepare the CAN frame sent periodically
29     tx_frame.id      = 0x100;    // 11-bit standard identifier
30     tx_frame.dlc     = 1;        // 1 data byte
31     tx_frame.data[0] = 0xAA;    // command: activate LED + buzzer
32     // on slave

```

```

30  /* USER CODE END 2 */
31
32  while (1)
33  {
34      /* USER CODE BEGIN 3 */
35
36      // Transmit the CAN frame
37      MCP2515_SendFrame(&tx_frame);
38
39      // Wait 3 seconds before the next transmission
40      HAL_Delay(3000);
41
42      /* USER CODE END 3 */
43  }
44  }

```

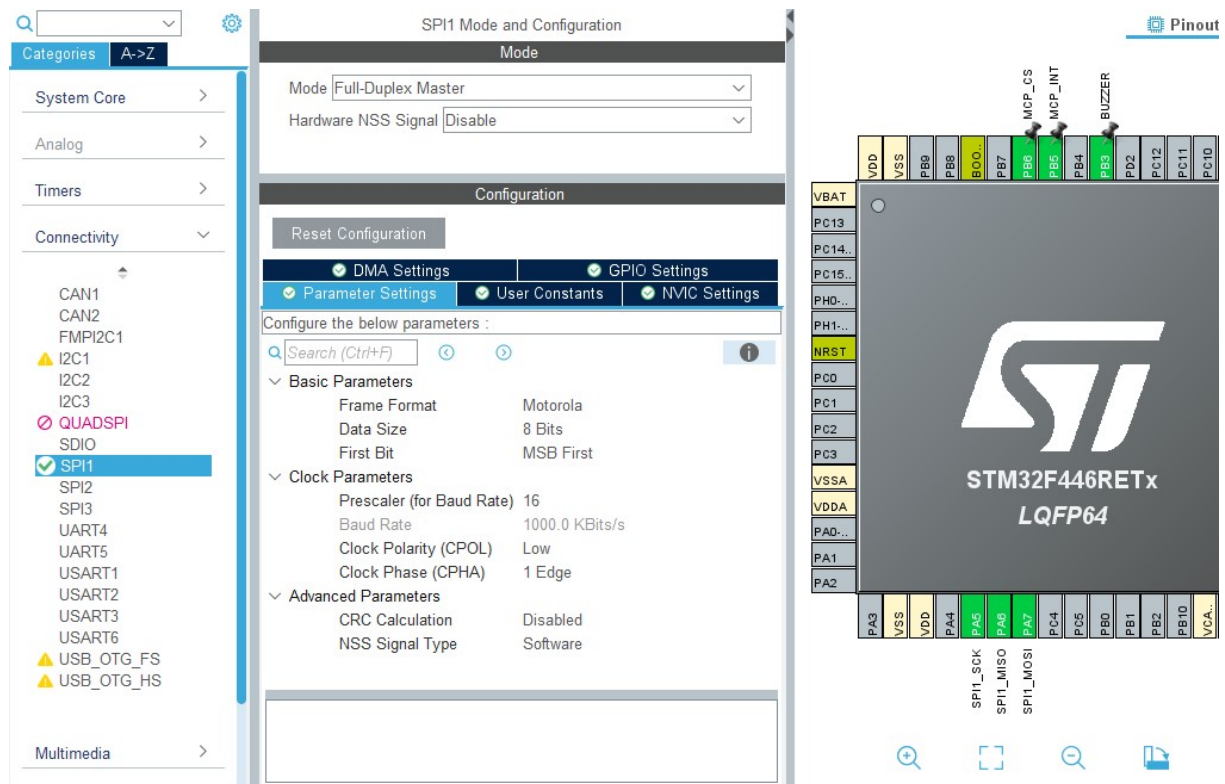
Listing 1: Master firmware

5.2 Slave Board Configuration

Create a new CubeMX project for the Slave board. The SPI1 and NVIC settings are *identical* to the Master. Additional GPIOs for the LED and buzzer are required.

5.2.1 SPI1 — Slave

Apply the same SPI1 settings as the Master (see Section 4.1.1).



5.2.2 GPIO — Slave

Check Fig. 5

Pin	Label	Mode
PB6	MCP_CS	GPIO Output
PB5	MCP_INT	GPIO EXTI5
PA10	LED_OUT	GPIO Output
PB3	BUZZER	GPIO Output

5.2.3 NVIC — Slave

Apply the same NVIC settings as the Master. Navigate to **System Core** → **NVIC** and enable:

- **EXTI line[9:5] interrupts**

The slave firmware is **interrupt-driven**: the MCP2515 asserts its INT pin low whenever a frame is received. This triggers the STM32 EXTI, which sets a flag (`can_irq_flag`) processed in the main loop.

Save the configuration: After finishing the configuration click **Ctrl + S**.

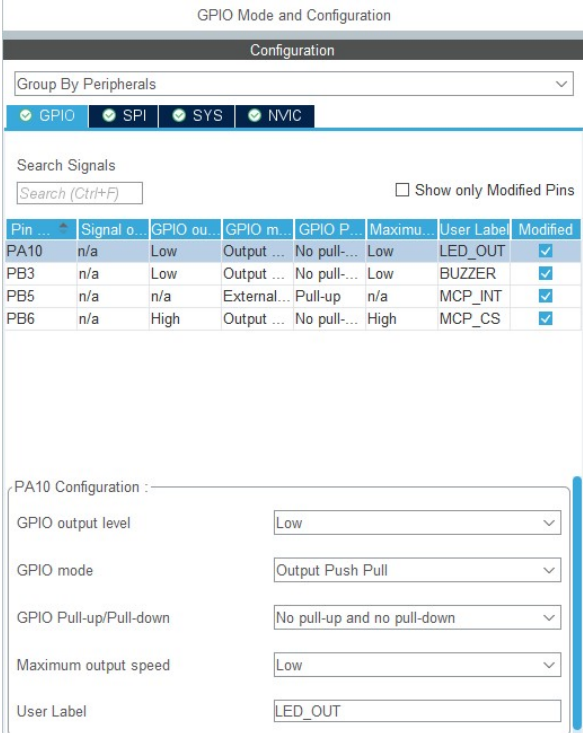
5.2.4 Complete main.c — Slave

Open the Slave **main.c** file and complete the application:

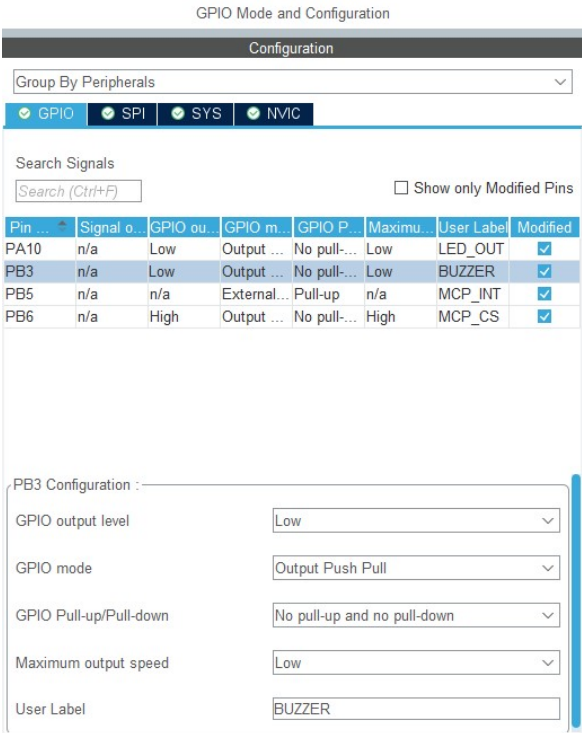
```

1  /* Includes -----*/
2  #include "main.h"
3
4  /* USER CODE BEGIN Includes */
5  #include "mcp2515.h"
6  /* USER CODE END Includes */
7
8  /* Private variables -----*/
9  SPI_HandleTypeDef hspi1;
10
11 /* USER CODE BEGIN PV */
12 CAN_Frame_t rx_frame;
13 volatile uint8_t can_irq_flag = 0; // set to 1 by EXTI when
14   MCP2515 signals reception
15 /* USER CODE END PV */
16
17 int main(void)
18 {
19     /* USER CODE BEGIN 2 */
20     // Let the MCP2515 stabilise after power-on

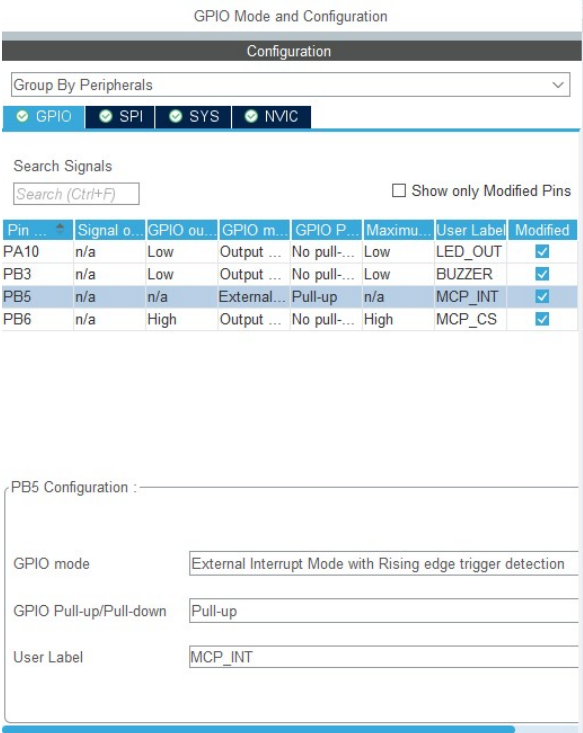
```



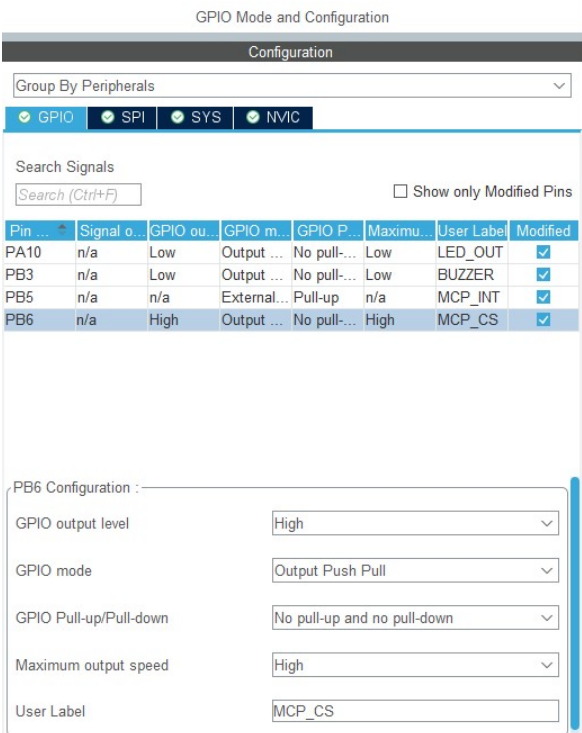
(a) PA10



(b) PB3



(c) PB5



(d) PB6

Figure 5: GPIO Configuration.

```
21     HAL_Delay(100);
22
23     // Initialise MCP2515: reset, configure 500 kbps, switch to
24     // Normal mode
25     MCP2515_Init();
26
27     /* USER CODE END 2 */
28
29     while (1)
30     {
31         /* USER CODE BEGIN 3 */
32
33         // Check whether the MCP2515 has raised an interrupt (frame
34         // received)
35         if (can_irq_flag)
36         {
37             can_irq_flag = 0; // clear the flag
38
39             // Confirm a frame is present in the receive buffer
40             if (MCP2515_CheckReceive())
41             {
42                 // Read the frame
43                 MCP2515_ReadFrame(&rx_frame);
44
45                 // Verify it is the expected master command
46                 if (rx_frame.id == 0x100 && rx_frame.data[0] == 0xAA)
47                 {
48                     // Blink LED and sound buzzer 3 times
49                     for (int i = 0; i < 3; i++)
50                     {
51                         HAL_GPIO_WritePin(LED_OUT_GPIO_Port, LED_OUT_Pin,
52                                             GPIO_PIN_SET);
53                         HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin,
54                                             GPIO_PIN_SET);
55                         HAL_Delay(150);
56                         HAL_GPIO_WritePin(LED_OUT_GPIO_Port, LED_OUT_Pin,
57                                             GPIO_PIN_RESET);
58                         HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin,
59                                             GPIO_PIN_RESET);
60                         HAL_Delay(150);
61                     }
62                 }
63             }
64         }
65
66         /* USER CODE END 3 */
67     }
68
69     /* USER CODE BEGIN 4 */
```

```

66  /**
67  * @brief EXTI callback: called automatically by HAL when a
68  *        falling edge
69  *        is detected on MCP_INT_Pin. Only sets the flag; no SPI
70  *        inside ISR.
71  */
72  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
73  {
74      if (GPIO_Pin == MCP_INT_Pin)
75      {
76          can_irq_flag = 1;
77      }
78  }
79  /* USER CODE END 4 */

```

Listing 2: Slave firmware

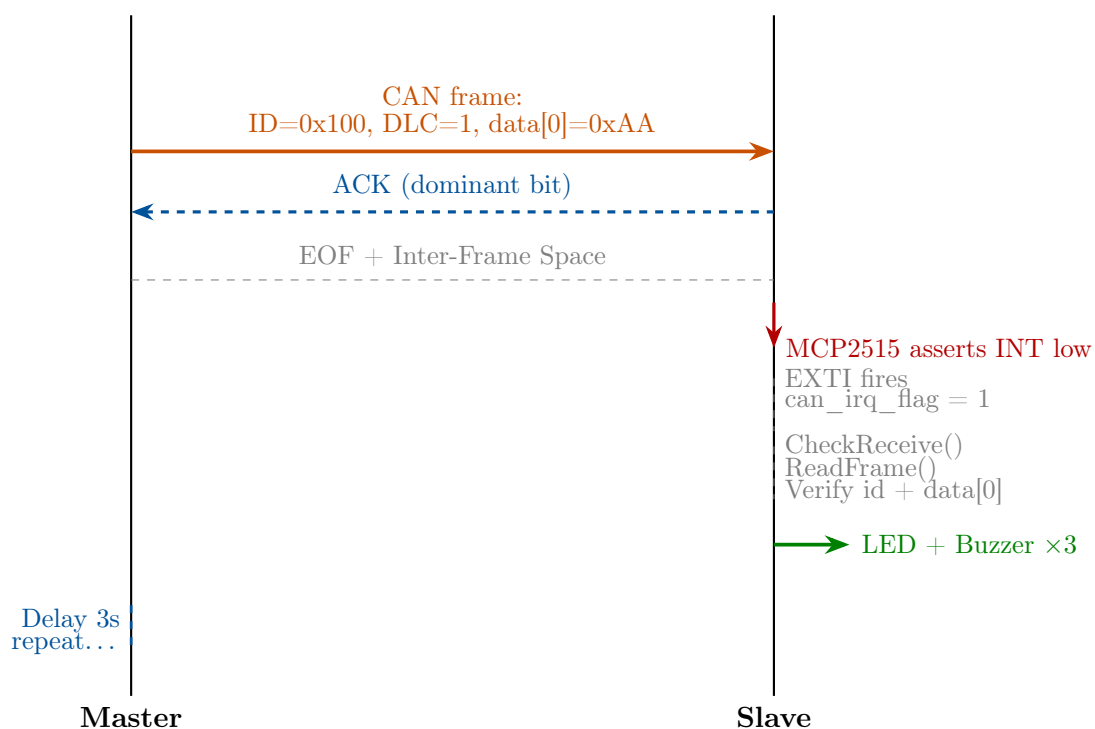
6 Communication Protocol

6.1 Frame Description

Only one CAN frame is exchanged in this application:

Direction	ID (hex)	DLC	Data[0]	Meaning
Master → Slave	0x100	1	0xAA	Activate LED + buzzer 3 times

6.2 Transaction Sequence



7 Project Validation

1. Connect the **Slave** STM32 to your PC via ST-LINK and flash the slave firmware. Disconnect it afterwards.
2. Connect the **Master** STM32 and flash the master firmware.
3. Power both boards and observe the slave: every 3 seconds the LED should blink and the buzzer should sound **three times**.

End of Guide